

10주차 파일

파일로 데이터를 영구적으로 저장하는 방법

학습목표

- 파일의 다양한 활용을 알아본다.
- 절대경로와 상대경로에 대해서 알아본다.
- 텍스트 파일과 바이너리 파일에 대해서 알아본다.
- CSV 파일 처리 방법에 대해서 알아본다.

파일의 필요성

- 파이썬에서 파일과 메모리는 데이터를 저장하고 처리하는 두 가지 주요 방식이다.
- 메모리
 - **RAM**(휘발성 저장소)에 데이터를 저장하며, 프로그램이 실행되는 동안만 데이터를 유지한다.
 - 프로그램이 종료되거나 시스템이 종료되면 데이터가 사라진다.
- 파일
 - 하드디스크나 **SSD**(비휘발성 저장소)에 데이터를 저장해 프로그램 종료 후에도 데이터를 유지한다.
 - 파일을 사용하면 프로그램이 종료된 후에도 데이터를 보존할 수 있다.
 - 예를 들어, 사용자의 설정, 게임 저장 데이터, 로그 등이 파일에 저장되어야 한다.

파일

- 파일은 데이터를 저장하고 관리하기 위한 컴퓨터 시스템의 기본 단위이다.
- 프로그램 실행 중 생성된 데이터를 저장하여 프로그램이 종료된 후에도 데이터를 유지하거나, 다른 프로그램이나 사용자와 데이터를 공유하기 위해 사용된다.
- 파이썬에서는 다양한 방식으로 파일을 생성, 읽기, 쓰기, 삭제할 수 있다.
- 파일의 기본 특징
 - 영구성: 파일은 비휘발성 저장 매체(예: 하드디스크, **SSD**)에 저장되어, 전원이 꺼져도 데이터를 유지한다.
 - 형식 다양성: 파일은 텍스트 파일(예: **.txt**, **.csv**)이나 바이너리 파일(예: 이미지, 동영상, 실행 파일) 등 다양한 형식으로 저장될 수 있다.
 - 구조화된 데이터 저장: 특정 형식(**JSON**, **XML**, **CSV** 등)을 사용하여 구조화된 데이터를 저장하고 다른 시스템에서 이를 쉽게 읽거나 변환할 수 있다.
 - 읽기/쓰기 가능: 파일은 데이터를 저장하는 쓰기(**write**)와 저장된 데이터를 불러오는 읽기(**read**) 작업을 통해 다룰 수 있다.

파일 처리

- 파일 처리를 위해 내장 함수와 `open()` 함수를 제공한다.
- 파일 열기
 - 파일을 읽거나 쓰기 전에 `open()` 함수를 사용하여 파일을 열어야 한다.
 - 모드는 다음과 같다.
 - 'r': 읽기 모드 (기본값).
 - 'w': 쓰기 모드 (파일이 없으면 생성, 있으면 덮어쓰).
 - 'a': 추가 모드 (파일이 없으면 생성, 있으면 뒤에 추가).
 - 'b': 바이너리 모드.
- 파일 읽기

```
with open("example.txt", "r") as file:  
    content = file.read()  
    print(content)
```

파일 처리

- 파일 쓰기

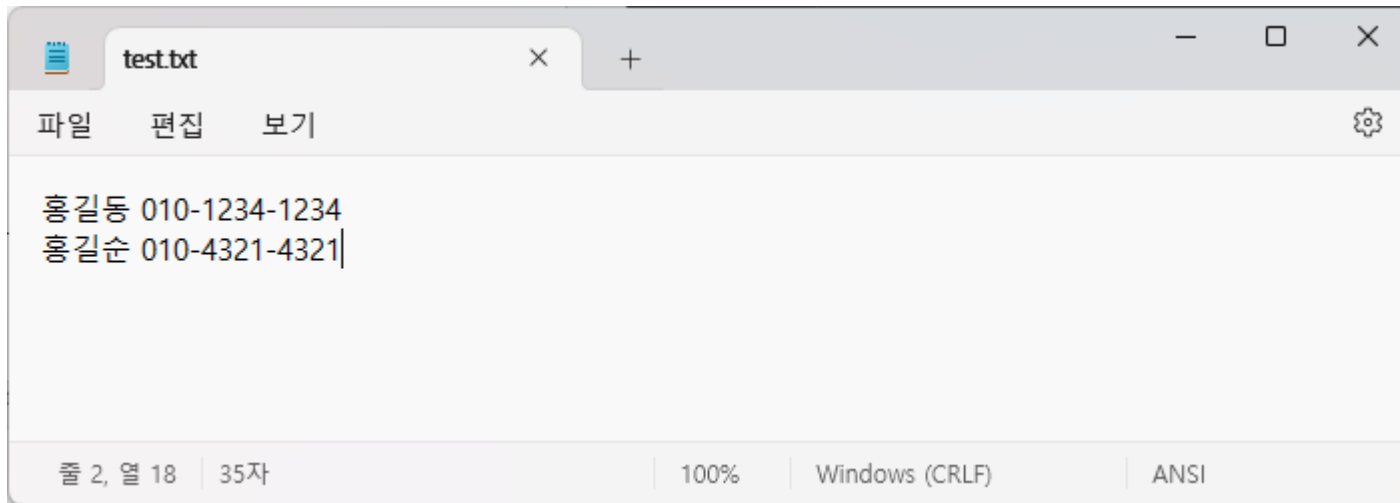
```
with open("example.txt", "w") as file:  
    file.write("Hello, World!")
```

- 파일 닫기

- 파일 작업이 끝나면 `close()`를 호출하거나 `with`문을 사용하여 자동으로 닫는다.

파일 열기 실습

- 메모장을 실행하고 다음과 같은 텍스트 파일을 작성한 후 **test.txt**로 저장한다.
- 주의: 파일 경로 관리파일을 읽고 쓰는 경로를 명확히 설정해야 한다. **절대 경로와 상대 경로**를 사용할 수 있다.



인코딩을 ANSI로 한다

절대 경로 vs 상대 경로

- 절대 경로 (Absolute Path)

- 파일 시스템의 최상위 루트 디렉터리(Windows에서는 드라이브 문자, Linux/Unix에서는 /)를 기준으로 파일이나 디렉터리의 위치를 완전히 지정한 경로이다.

- Windows:

```
C:\Users\username\Documents\example.txt
```

- Linux/Unix:

```
/home/username/Documents/example.txt
```

- 장점

- 경로가 명확하기 때문에 파일 위치를 정확히 지정할 수 있다.

- 단점

- 경로가 길어질 수 있으며, 다른 시스템에서 경로가 달라지면 사용할 수 없게 된다.

절대 경로 vs 상대 경로

- 상대 경로 (Relative Path)

- 정의 현재 작업 디렉터리(현재 프로그램이 실행 중인 위치)를 기준으로 파일이나 디렉터리의 위치를 지정한 경로이다.
- 현재 디렉터리를 기준으로 경로를 지정하므로, 현재 디렉터리가 바뀌면 경로도 바뀔 수 있다.
- .(현재 디렉터리)와 ..(상위 디렉터리)를 사용할 수 있다.
- 현재 디렉터리가 /home/username/일 때

Documents/example.txt

- 상위 디렉터리로 이동

../other_folder/example.txt

- 장점
 - 경로가 짧고, 코드가 실행되는 환경에 따라 유연하게 사용할 수 있다.
- 단점
 - 현재 디렉터리에 의존하므로, 디렉터리가 변경되면 파일을 찾지 못할 가능성이 있다.

파일에서 데이터 읽기

1. 파일을 열다.
2. 파일에서 데이터를 읽거나 쓸 수 있다.
3. 파일과 관련된 작업이 모두 종료되면 파일을 닫아야 한다.

```
# 파일 열기
file = open("test.txt", "r")

# 파일 내용 읽기
content = file.read()

# 내용 출력
print(content)

# 파일 닫기
file.close()
```

```
# with문으로 파일 열기
with open("test.txt", "r") as file:
    # 파일 내용 읽기
    content = file.read()

# 내용 출력
print(content)
```

with문 사용 (권장)

```
홍길동 010-1234-1234
홍길순 010-4321-4321
```

파일에서 읽기

- 한 줄씩 읽기 (readline())
 - 파일의 한 줄씩 읽어서 처리한다.

```
with open("test.txt", "r") as file:  
    line = file.readline() # 첫 번째 줄 읽기  
    while line: # 더 이상 읽을 줄이 없으면 종료  
        print(line.strip()) # 줄바꿈 제거 후 출력  
        line = file.readline() # 다음 줄 읽기
```

```
홍길동 010-1234-1234  
홍길순 010-4321-4321
```

- 장점: 메모리 효율적, 파일을 순차적으로 처리 가능.
- 단점: 특정 라인만 접근하기 어렵고 반복 처리가 필요.

파일에서 읽기

- 모든 줄을 리스트로 읽기 (`readlines()`)
 - 파일의 모든 줄을 읽어 리스트 형태로 반환한다.

```
with open("test.txt", "r") as file:  
    lines = file.readlines() # 각 줄이 리스트의 요소로 저장됨  
    for line in lines:  
        print(line.strip()) # 줄바꿈 제거 후 출력
```

```
홍길동 010-1234-1234  
홍길순 010-4321-4321
```

- 장점: 각 줄을 리스트로 다룰 수 있어 특정 줄에 쉽게 접근 가능.
- 단점: 파일이 클 경우 메모리 문제 발생 가능.

파일에서 읽기

- 파일을 반복문으로 읽기
 - 파일 객체는 반복 가능(iterable)하므로 for 루프를 사용하여 한 줄씩 읽을 수 있다.

```
with open("test.txt", "r") as file:  
    for line in file: # 파일 객체를 반복  
        print(line.strip()) # 줄바꿈 제거 후 출력
```

```
홍길동 010-1234-1234  
홍길순 010-4321-4321
```

- 장점: 메모리 효율적, 코드가 간결함.
- 단점: 파일이 순차적으로만 처리됨.

인코딩

- 텍스트 파일은 특정 인코딩 방식(예: UTF-8, EUC-KR 등)으로 저장되며, 이를 올바르게 읽거나 쓰기 위해 파일의 인코딩 방식을 지정해야 한다.
- 파일 쓰기 시 인코딩 지정
 - 파일을 저장할 때 인코딩을 지정하면 텍스트를 해당 인코딩 방식으로 저장할 수 있다.

```
# UTF-8 인코딩으로 파일 쓰기
with open("example.txt", "w", encoding="utf-8") as file:
    file.write("안녕하세요")
```

- 파일 읽기 시 인코딩 지정
 - 파일을 읽을 때 저장된 인코딩 방식에 맞게 지정해야 한다.

```
# UTF-8 인코딩으로 파일 읽기
with open("example.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print(content)
```

인코딩 관련 주요 옵션

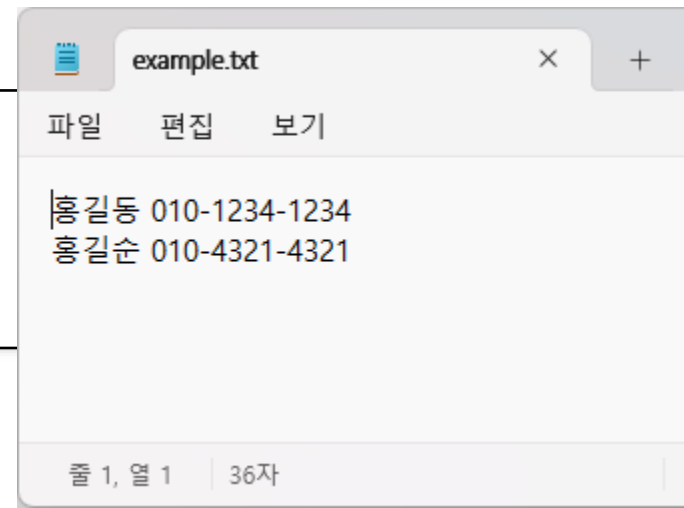
- encoding 매개변수
 - 파일의 인코딩 방식을 지정한다.
 - 지원하는 주요 인코딩
 - utf-8: 대부분의 언어를 지원하는 유니코드 표준.
 - euc-kr: 한국어 인코딩.
 - cp949: Windows 한국어 인코딩.
 - latin-1: 유럽 언어 지원.
 - ascii: 기본 영어 문자.
- EUC-KR 같은 다른 인코딩을 사용할 경우

```
# EUC-KR로 파일 읽기
with open("example.txt", "r", encoding="euc-kr") as file:
    content = file.read()
    print(content)
```

파일에 데이터 쓰기

- 파이썬에서 파일에 데이터를 쓰는 작업은 `open()` 함수와 쓰기 관련 메서드를 활용하여 수행한다.
- 파일을 열 때 모드를 지정하여 데이터를 새로 쓰거나 기존 데이터에 추가할 수 있다.
- 파일을 다룰 때는 파일 모드와 데이터 처리 방식을 적절히 선택하는 것이 중요하다.
- 파일 쓰기 기본 원리
 - 파이썬에서 파일 쓰기는 다음과 같은 절차를 따른다
 1. 파일 열기: `open()` 함수로 파일을 열고 쓰기 모드 지정.
 2. 데이터 쓰기: `write()` 또는 `writelines()` 메서드 사용.
 3. 파일 닫기: `close()` 메서드 또는 `with`문으로 자동 닫기.

```
outfile = open("example.txt", "w")
outfile.write("홍길동 010-1234-1234\n")
outfile.write("홍길순 010-4321-4321\n")
outfile.close()
```



파일에 데이터 쓰기

- 파일에 새 데이터 쓰기 ('w' 모드)
 - 파일이 없으면 생성하고, 있으면 기존 내용을 삭제한 후 새 데이터를 쓴다.

```
with open("example.txt", "w", encoding="utf-8") as file:  
    file.write("안녕하세요, 파이썬입니다.\n")  
    file.write("파일 쓰기를 배우고 있습니다.\n")
```

- 파일에 데이터 추가 ('a' 모드)
 - 기존 파일의 내용을 유지하면서, 새 데이터를 추가로 작성한다.

```
with open("example.txt", "a", encoding="utf-8") as file:  
    file.write("이 줄은 기존 내용 뒤에 추가되었습니다.\n")
```

- 여러 줄 쓰기 (writelines())
 - 리스트로 된 여러 줄의 데이터를 한 번에 작성할 수 있다.

```
lines = ["첫 번째 줄\n", "두 번째 줄\n", "세 번째 줄\n"]  
with open("example.txt", "w", encoding="utf-8") as file:  
    file.writelines(lines)
```

도전 문제

- 주어진 텍스트 파일에서 특정 단어를 읽고, 그 단어가 몇 번 등장했는지 계산하는 프로그램을 작성하시오.
- 문제 조건
 - 파일 이름은 `text.txt`로 가정한다.
 - 파일 내용(`text.txt`)

Hello, world! Hello Python. Python is fun. Hello again!

- 사용자로부터 찾고자 하는 단어를 입력받는다.
- 파일에서 해당 단어가 몇 번 등장했는지 출력한다.
- 단어 비교는 대소문자를 구분하지 않는다.
- 단어와 단어 사이의 구분은 공백과 구두점(예: ,, ,, !) 등을 고려한다.
- 해당 기능을 `count_word_in_file` 함수로 정의한다.

찾고자 하는 단어를 입력하세요: hello
단어 "hello"는 3번 등장했습니다.

도전 문제 정답

```
import string
# 파일에서 단어를 읽고 등장 횟수를 계산하는 함수
def count_word_in_file(file_name, target_word):
    with open(file_name, "r") as file:
        content = file.read()
        # 모든 문자를 소문자로 변환
        content = content.lower()
        target_word = target_word.lower()
        # 구두점 제거
        translator = str.maketrans("", "", string.punctuation)
        content = content.translate(translator)
        # 단어 리스트 생성
        words = content.split()
        # 단어 등장 횟수 계산
        count = words.count(target_word)
    return count

# 사용자 입력 및 결과 출력
file_name = "text.txt"
target_word = input("찾고자 하는 단어를 입력하세요: ")
result = count_word_in_file(file_name, target_word)
print(f'단어 "{target_word}"는 {result}번 등장했습니다.')
```

파일의 주요 유형

- 텍스트 파일
 - 사람이 읽을 수 있는 형식의 파일.
 - 예: .txt, .csv, .json.
- 바이너리 파일
 - 사람이 읽을 수 없는 형식의 파일로, 이진 데이터로 저장됨.
 - 예: 이미지, 오디오, 비디오, 실행 파일.

바이너리 파일 처리

- 바이너리 파일 처리는 텍스트가 아닌 이진 데이터를 읽거나 쓰는 작업을 말한다.
- 바이너리 파일은 이미지, 동영상, 오디오, 실행 파일 등 텍스트로 표현되지 않는 데이터를 포함하므로, 처리할 때는 텍스트 모드가 아닌 바이너리 모드를 사용해야 한다.
- 바이너리 파일 열기
 - 바이너리 모드는 'b'를 사용하여 파일을 열 때 지정한다.
 - 주로 'rb'(읽기) 또는 'wb'(쓰기) 모드를 사용한다.
- 파일 모드
 - 'rb' 읽기 전용 바이너리 모드.
 - 'wb' 쓰기 전용 바이너리 모드. 기존 데이터 삭제.
 - 'ab' 추가 모드. 기존 데이터 뒤에 추가.
 - 'rb+' 읽기/쓰기 바이너리 모드. 기존 내용 유지.

바이너리 파일 쓰기

- 전체 데이터 쓰기
 - 바이너리 데이터를 파일에 저장하는 예제:

```
data = b'\x00\x01\x02\x03\x04' # 이진 데이터
```

```
with open("output.bin", "wb") as file:  
    file.write(data) # 데이터 쓰기
```

- 데이터 추가
 - 기존 바이너리 파일의 끝에 데이터를 추가

```
data = b'\x05\x06\x07'
```

```
with open("output.bin", "ab") as file:  
    file.write(data) # 기존 파일 끝에 추가
```

바이너리 파일 읽기

- 전체 읽기
 - 파일의 모든 내용을 한 번에 읽는 예제

```
with open("example.bin", "rb") as file:  
    data = file.read() # 파일 전체를 읽음  
    print(data) # 바이너리 데이터 출력
```

- 특정 크기만큼 읽기
 - 큰 파일을 처리할 때는 데이터를 일정 크기씩 나누어 읽는다.

```
with open("example.bin", "rb") as file:  
    while chunk := file.read(1024): # 1024 바이트씩 읽음  
        print(chunk) # 각 청크(chunk)를 출력
```

바이너리 파일 읽기

- 한 줄씩 읽기
 - 바이너리 파일에서 줄 단위로 읽을 수도 있다.

```
with open("example.bin", "rb") as file:  
    for line in file:  
        print(line)
```


도전 문제

- 파일을 복사하는 프로그램을 작성해보자. 파일의 이름은 사용자가 입력하도록 하자.

입력 파일 이름: example.bin
출력 파일 이름: temp.bin

도전 문제 정답

```
# 입력 파일 이름과 출력 파일 이름을 받는다.  
source_file = input("입력 파일 이름: ");  
destination_file = input("출력 파일 이름: ");  
  
with open(source_file, "rb") as src, open(destination_file, "wb") as dest:  
    while chunk := src.read(1024): # 1024 바이트씩 읽어서  
        dest.write(chunk) # 그대로 기록
```

바이너리 데이터와 struct 모듈

- 바이너리 데이터를 다룰 때, 숫자나 텍스트를 특정 형식으로 변환하거나 해석하려면 `struct` 모듈을 사용할 수 있다.
- 데이터 패킹과 언패킹
 - `struct.pack()`을 사용해 데이터를 바이너리 형식으로 변환하고, `struct.unpack()`으로 다시 해석한다.

```
import struct

# 정수와 실수를 바이너리 데이터로 변환
packed_data = struct.pack("i f", 42, 3.14) # 'i': 정수, 'f': 실수
print(packed_data) # b'\x00\x00\x00\xc3\xf5H@'

# 바이너리 데이터를 원래 값으로 복원
unpacked_data = struct.unpack("i f", packed_data)
print(unpacked_data) # (42, 3.14)
```

바이너리 데이터와 struct 모듈

- 파일에 데이터 저장 및 읽기
 - 바이너리 데이터를 파일에 저장하고 다시 읽는다.

```
import struct

# 데이터 패킹
data = struct.pack("i f", 42, 3.14)

# 파일에 쓰기
with open("data.bin", "wb") as file:
    file.write(data)

# 파일에서 읽기
with open("data.bin", "rb") as file:
    read_data = file.read()
    unpacked = struct.unpack("i f", read_data)
    print(unpacked) # (42, 3.14)
```

도전 문제

- 특정한 디렉토리 안의 파일 중에서 "Python"을 포함하고 있는 줄이 있으면 파일의 이름과 해당 줄을 출력한다.
- 주의사항: 텍스트 파일만 들어 있는 디렉토리에서 실행하여야 합니다. 그렇지 않으면 인코딩 오류가 발생합니다.

```
file.py : if "Python" in e:  
summary.txt : The joy of coding Python should be in seeing short  
summary.txt : Python is executable pseudocode.
```

도전 문제 정답

```
import os
arr = os.listdir()

for f in arr:
    infile = open(f, "r", encoding="utf-8")    # 텍스트 파일이 아니면 오류가 발생한다.
    for line in infile:
        e = line.rstrip()    # 오른쪽 줄바꿈 문자를 없앤다.
        if "Python" in e:
            print(f, ":", e)
    infile.close()
```

CSV 파일 처리하기

- CSV (Comma-Separated Values) 파일은 데이터를 텍스트 형식으로 저장하는 가장 간단한 파일 형식 중 하나로, 파이썬에서 처리하기 매우 쉽다.
- 파이썬은 CSV 파일을 처리하기 위해 기본적으로 `csv` 모듈을 제공하며, 데이터를 효율적으로 읽고 쓰기 위한 다양한 방법을 지원한다.

- 예제: `example.csv`

```
이름,나이,직업  
홍길동,25,개발자  
김철수,30,디자이너
```

- 출력

```
['이름', '나이', '직업']  
['홍길동', '25', '개발자']  
['김철수', '30', '디자이너']
```

CSV 파일 읽기

- 기본 읽기
 - 파일을 한 줄씩 읽어서 리스트 형태로 가져올 수 있다.

```
import csv

# CSV 파일 읽기
with open("example.csv", "r", encoding="utf-8") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row) # 각 행이 리스트 형태로 출력됨
```


CSV 파일 읽기

- 헤더 포함 읽기
 - 첫 번째 행을 헤더로 처리하고 데이터를 딕셔너리 형태로 가져온다.

```
import csv

with open("example.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row) # 각 행이 딕셔너리 형태로 출력됨
```

CSV 파일 쓰기

- 기본 쓰기
 - 리스트 데이터를 CSV 형식으로 저장한다.

```
import csv

# 데이터
data = [
    ["이름", "나이", "직업"],
    ["홍길동", 25, "개발자"],
    ["김철수", 30, "디자이너"]
]

# CSV 파일 쓰기
with open("output.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(data) # 리스트 데이터를 한 번에 작성
```

CSV 파일 쓰기

- 딕셔너리 데이터 쓰기
 - 딕셔너리 데이터를 CSV 형식으로 저장할 때는 `csv.DictWriter`를 사용한다.

```
import csv

# 데이터
data = [
    {"이름": "홍길동", "나이": 25, "직업": "개발자"},
    {"이름": "김철수", "나이": 30, "직업": "디자이너"}
]

# CSV 파일 쓰기
with open("output_dict.csv", "w", encoding="utf-8", newline="") as file:
    fieldnames = ["이름", "나이", "직업"]
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader() # 헤더 작성
    writer.writerows(data) # 딕셔너리 데이터 작성
```

CSV 파일 쓰기

- 특정 구분자
 - 사용다른 구분자를 사용하는 CSV 파일 작성.

```
import csv

data = [
    ["이름", "나이", "직업"],
    ["홍길동", 25, "개발자"],
    ["김철수", 30, "디자이너"]
]

with open("output_semicolon.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file, delimiter=";")
    writer.writerows(data)
```

CSV 파일 수정

- CSV 파일 데이터를 읽어 특정 값을 수정한 뒤 다시 저장할 수 있다.

```
import csv

# 수정할 데이터 읽기
rows = []
with open("example.csv", "r", encoding="utf-8") as file:
    reader = csv.reader(file)
    for row in reader:
        rows.append(row)

# 데이터 수정 (예: 나이를 1 증가)
for row in rows[1:]: # 첫 번째 행(헤더)을 제외하고 처리
    row[1] = str(int(row[1]) + 1)

# 수정된 데이터 다시 저장
with open("example.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(rows)
```

도전 문제

- 주어진 CSV 파일(students.csv)에는 학생들의 이름, 점수, 학점 정보가 저장되어 있다. 이 파일을 이용한 학점 계산과 최고 점수를 출력하고 학점이 추가된 파일을 새로 만드는 프로그램을 작성해보자!

- 파일 내용 (students.csv)

```
이름,점수,학점  
홍길동,85,  
김철수,92,  
이영희,76,  
박지민,59,  
최준혁,100,
```

- 학점 계산

90~100: A	80~89 : B	70~79 : C	60~69 : D	0~59 : F
-----------	-----------	-----------	-----------	----------

- 새로운 CSV 파일 저장: 학점이 추가된 데이터를 updated_students.csv 파일에 저장
- 최고 점수 출력: 프로그램 종료 시 최고 점수 학생의 이름과 점수를 출력

도전 문제 정답

```
import csv

def calculate_grade(score):
    if 90 <= score <= 100:
        return 'A'
    elif 80 <= score < 90:
        return 'B'
    elif 70 <= score < 80:
        return 'C'
    elif 60 <= score < 70:
        return 'D'
    else:
        return 'F'

# 1. CSV 파일 읽기
students = []
with open("students.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        students.append(row)
```

뒷장에 계속

도전 문제 정답

2. 학점 계산 및 데이터 업데이트

```
for student in students:
```

```
    score = int(student["점수"])
```

```
    student["학점"] = calculate_grade(score)
```

3. 최고 점수 계산

```
top_student = max(students, key=lambda x: int(x["점수"]))
```

4. 업데이트된 데이터 저장

```
with open("updated_students.csv", "w", encoding="utf-8", newline="") as file:
```

```
    fieldnames = ["이름", "점수", "학점"]
```

```
    writer = csv.DictWriter(file, fieldnames=fieldnames)
```

```
    writer.writeheader()
```

```
    writer.writerows(students)
```

5. 최고 점수 출력

```
print(f'최고 점수 학생: {top_student["이름"]}, 점수: {top_student["점수"]})
```


Q & A